



Data Analytics Project: **KC_HOUSE_DATA**

Submitted By: Team 4

- 1.Ambika S M
- 2.yamini
- 2.Ram Sreekar
- 3.Venkata Karthik
- 4.Shashi Kumar V

Dataset Used: kc_house_data.csv

This document outlines a structured, end-to-end data analysis project based on the kc_house_data dataset. The project is divided into five scenarios and includes important features such as house price, number of bedrooms, number of bathrooms, living area (sqft_living), and other property-related attributes. This dataset serves as the foundation for performing data cleaning, analysis, and visualization tasks to understand housing trends and pricing patterns.

Project Goal: The main goal of this project is to analyse house sales data to identify patterns, trends, and insights related to pricing, bedroom distribution, and property categories.

The project focuses on using data analysis techniques to clean raw data, visualize patterns, and derive meaningful business insights.

The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (`np.diff`).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts).

Project Coverage

Category	Skills Covered
Data Preprocessing	Data Loading, Missing Value Handling (Mean/Mode), Type Conversion, Column Removal
Pandas Operations	Grouping, Filtering, Aggregation, Feature Engineering (score_category, editors_choice conversion)
NumPy Calculations	Array conversion, Yearly score growth calculation (<code>np.diff</code>)
Visualization	Line Chart, Bar Chart, Pie Chart, Histogram, Stacked Bar Chart, Saving Plots

Scenario 1: Data Loading & Basic Cleaning (🟡 Easy)

Description:

In this scenario, the `kc_house_data` dataset is loaded using Pandas and explored to understand its structure. The first few rows and column names are displayed to get an overview of the data. Important columns such as `bedrooms`, `bathrooms`, `sqft_living`, and `price` are checked for missing values. Data types are converted into numeric format where required. Missing values are handled using statistical methods like mean and mode. This ensures the dataset is clean and ready for further analysis.

Task	Description
1.	Load the <code>kc_house_data.csv</code> dataset using Pandas.
2.	Display: ○ First 5 rows ○ Column names
3.	Check for missing values in: ○ <code>bedrooms</code> ○ <code>bathrooms</code> ○ <code>sqft_living</code> ○ <code>price</code>
4.	Fill missing values: ○ <code>bedrooms</code> → use mode ○ <code>bathrooms</code> → use mean ○ <code>sqft_living</code> → use mean ○ <code>price</code> → use mean
5.	Convert <code>Height</code> column to numeric if required.

```

First 5 rows:
   id      date      price  bedrooms  ...      lat      long  sqft_living15  sqft_lot15
0  7129300520  20141013T000000  221900.0      3  ...  47.5112 -122.257      1340      5650
1  6414100192  20141209T000000  538000.0      3  ...  47.7210 -122.319      1690      7639
2  5631100400  20150225T000000  180000.0      2  ...  47.7379 -122.233      2720      8062
3  2487200875  20141209T000000  604000.0      4  ...  47.5208 -122.393      1360      5000
4  1954400510  20150218T000000  510000.0      3  ...  47.6168 -122.045      1800      7503

[5 rows x 21 columns]
-----
Column names:
Index(['id', 'date', 'price', 'bedrooms', 'bathrooms', 'sqft_living',
       'sqft_lot', 'floors', 'waterfront', 'view', 'condition', 'grade',
       'sqft_above', 'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode',
       'lat', 'long', 'sqft_living15', 'sqft_lot15'],
      dtype='str')
-----
Missing values BEFORE cleaning:
bedrooms      0
bathrooms     0
sqft_living    0
price         0
dtype: int64
-----

```

Figure 1.1: Displaying head(), column functions

Conclusion:

The dataset was successfully loaded and explored using Pandas. Missing values in key columns were identified and handled using appropriate techniques. Data types were standardized to ensure consistency. The cleaning process improved the reliability of the dataset. This prepared the data for further visualization and analysis.

Overall, the dataset is now ready for the next steps.

Scenario 2: Line Graph + Save (🟡 Easy)

Description:

In this scenario, a subset of the dataset is selected to analyze house price trends. The id and price columns are extracted, and only the first 10 rows are considered for simplicity. The price values are converted into a NumPy array for numerical processing. A line graph is plotted to visualize how prices vary across these entries. Proper titles and labels are added for clarity. The graph is saved for future reference.

Task	Description
1.	Select: ○ id ○ price
2.	Take First 10 rows only.
3.	Convert price into a NumPy array
4.	Plot a line graph: ○ X-axis → index (0–9) ○ Y-axis → Price
5.	Add title and labels.
6.	Save the graph: <code>plt.savefig("house_prices_line.png")</code>

Graph:

A line graph is created showing the variation of house prices for the first 10 records. The X-axis represents the index (0–9), and the Y-axis represents the price. This graph helps in understanding short-term price variation patterns.

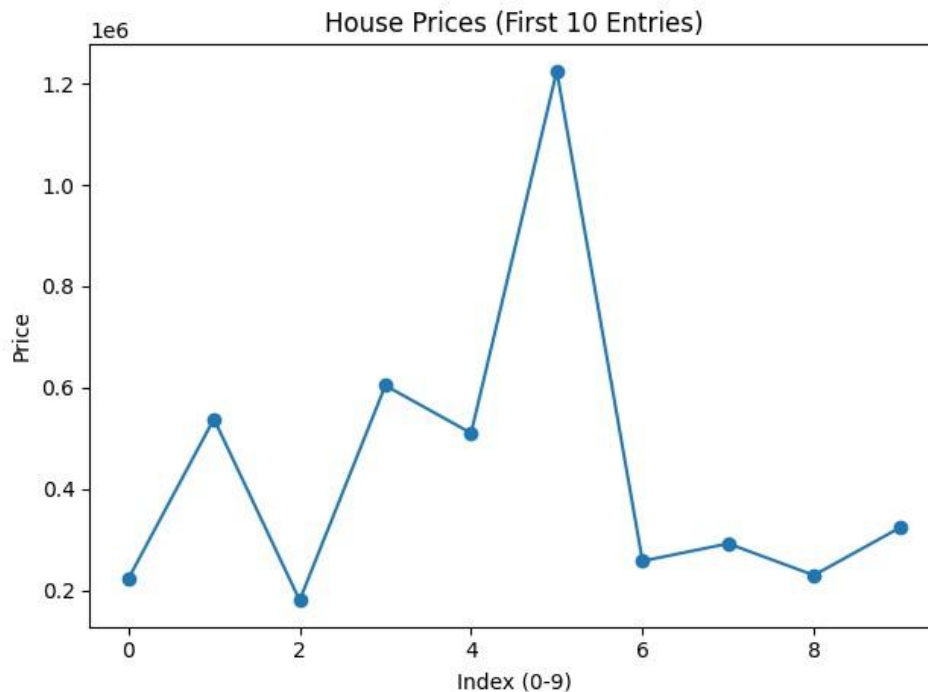


Figure 2.1: Line Graph of First 10 House Prices

Conclusion:

The line graph provides a clear visualization of price changes across selected houses. It helps identify variations and trends in the initial dataset. Converting data to NumPy arrays improves efficiency in plotting. The graph is simple yet effective for understanding price distribution. Saving the graph allows for easy sharing and reporting. This scenario builds a foundation for more advanced visualizations.

Scenario 3: Filtering + Bar Chart + Save (🌀 Medium)

Description:

In this scenario, the focus is on analyzing expensive houses in the dataset. Houses with prices greater than 1,000,000 are filtered to study high-value properties. The number of houses is counted based on bedroom categories. The top bedroom categories are selected for better visualization. The results are converted into NumPy arrays for plotting. A bar chart is created to show the relationship between bedroom count and number of expensive houses.

Task	Description
1.	Filter houses where: ○ price > 1000000
2.	Count number of houses per: ○ bedrooms
3.	Select top bedroom categories.
4.	Convert results to NumPy arrays.
5.	Plot a bar chart: ○ X-axis → Bedrooms ○ Y-axis → count
6.	Rotate labels if needed
7.	Save graph: <code>plt.savefig("expensive_houses_bar.png")</code>

Graph:

A bar chart is plotted where the X-axis represents the number of bedrooms, and the Y-axis represents the count of expensive houses. This graph highlights which bedroom categories are most common among high-priced properties.



Figure 3.1: Bar Graph of Expensive Houses per Bedroom Count

Conclusion:

The bar chart clearly shows the distribution of expensive houses across different bedroom categories. It helps identify which types of houses dominate the luxury segment. Filtering allows focused analysis on high-value properties. The visualization makes comparison easier and more effective. This scenario provides insights into market trends for premium housing. It also supports better decision-making in real estate analysis.

Scenario 4: Pie Chart (Region Distribution) + Save (🌀 Medium)

Description:

In this Scenario, the pie chart represents the distribution of houses across different regions, showing how properties are spread geographically. Each slice indicates the proportion of houses in a specific region relative to the total dataset. Larger slices highlight regions with higher numbers of houses, making them dominant areas. This visualization helps in quickly comparing regional contributions. The chart also includes percentage labels, making the distribution easy to understand at a glance.

Task	Description
1.	Count the number of houses by: ◦ bedrooms
2.	Select top 5 bedroom categories
3.	Prepare labels and values.
4.	Plot a pie chart.
5.	Add percentage labels.
6.	Save graph: <code>plt.savefig("bedroom_distribution.png")</code>

Graph :

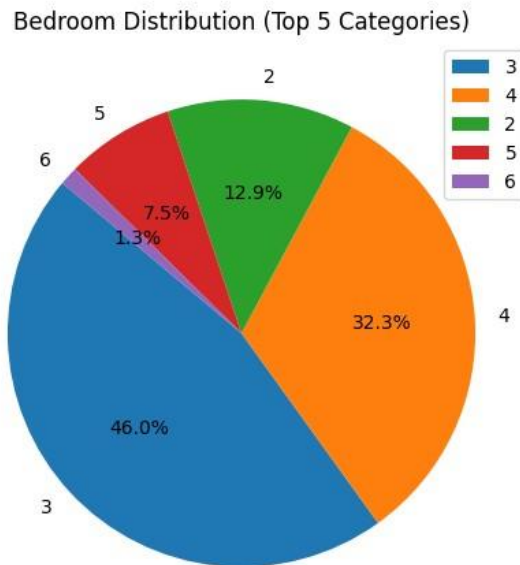


Fig 4.1: Pie Chart for Bedroom Distribution.

Conclusion:

The pie chart clearly shows how houses are distributed across different regions. It highlights the regions with the highest concentration of properties compared to others. A few regions dominate the dataset, while some have relatively fewer houses. This indicates an uneven distribution of housing across locations. Overall, the chart helps in understanding regional trends and identifying key areas in the housing market.

Scenario 5: Advanced Analysis + Multiple Graphs (🟡 Hard)

Description:

This scenario involves advanced analysis of the house sales dataset to uncover deeper insights. A new feature called Price Category is created to classify houses into different price ranges. NumPy is used to analyse changes in house prices and identify trends. Multiple graphs such as line graphs, stacked bar charts, and histograms are used to visualize patterns in the data. Overall, this helps in understanding pricing behaviour and distribution of houses more effectively.

Task	Description
1.	Feature Creation Create a new column: Price Category • price $\geq$ 1000000 $\rightarrow$ "Luxury" • 500000 – 999999 $\rightarrow$ "Mid Range" • < 500000 $\rightarrow$ "Affordable"
2.	NumPy Usage 1. Convert price column to a NumPy array. 2. Calculate price differences using: np.diff()
3.	Visualizations Line Graph 4. Plot Price trend for all houses Stacked Bar Chart
	5. Show count of Price Category per Bedrooms. Histogram 6. Plot distribution of: ○ price
4.	Save All Graphs plt.savefig("price_trend.png") plt.savefig("price_category_stacked.png") plt.savefig("price_distribution.png")

Graph :

1. Line Graph :

The line graph shows the overall trend of house prices across the dataset. It helps identify fluctuations and patterns in pricing. This makes it easier to observe whether prices are increasing or varying over time/index.

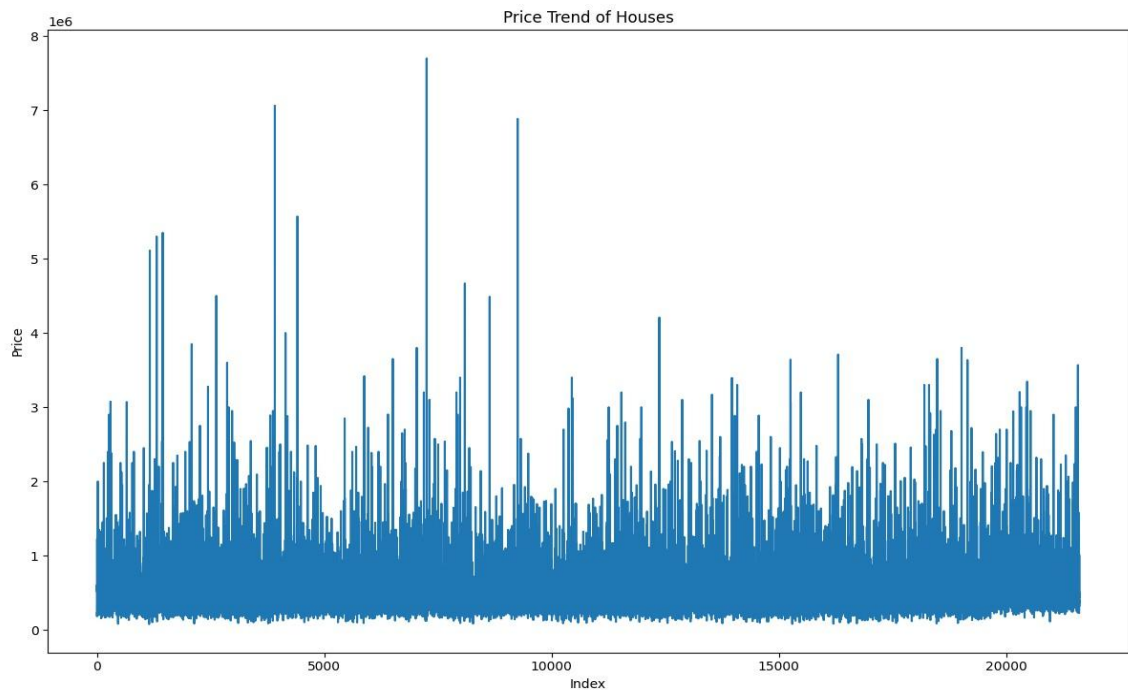


Fig 5.1: Line Graph for Price Trend of Houses

2. Stacked Bar Chart :

The stacked bar chart displays the distribution of price categories across different bedroom types. It helps compare how Affordable, Mid Range, and Luxury houses are spread among bedroom categories. This visualization highlights relationships between house size and pricing.

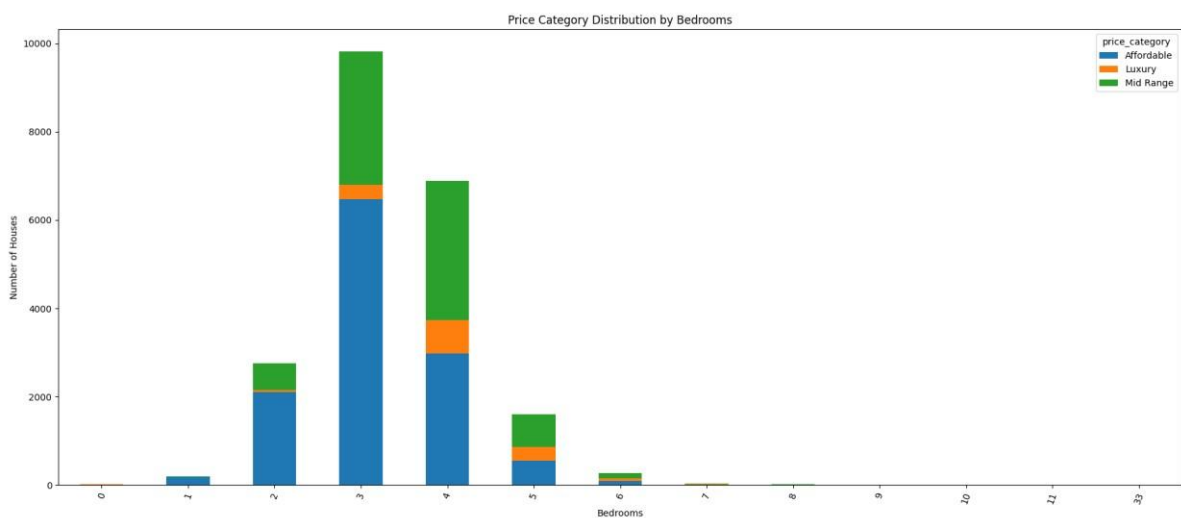


Fig 5.2: Stacked Bar Chart for Price Category Per Bedrooms.

3. Histogram :

The histogram shows the distribution of house prices across different ranges. It helps identify how frequently certain price ranges occur. This makes it easier to understand the spread and concentration of prices.

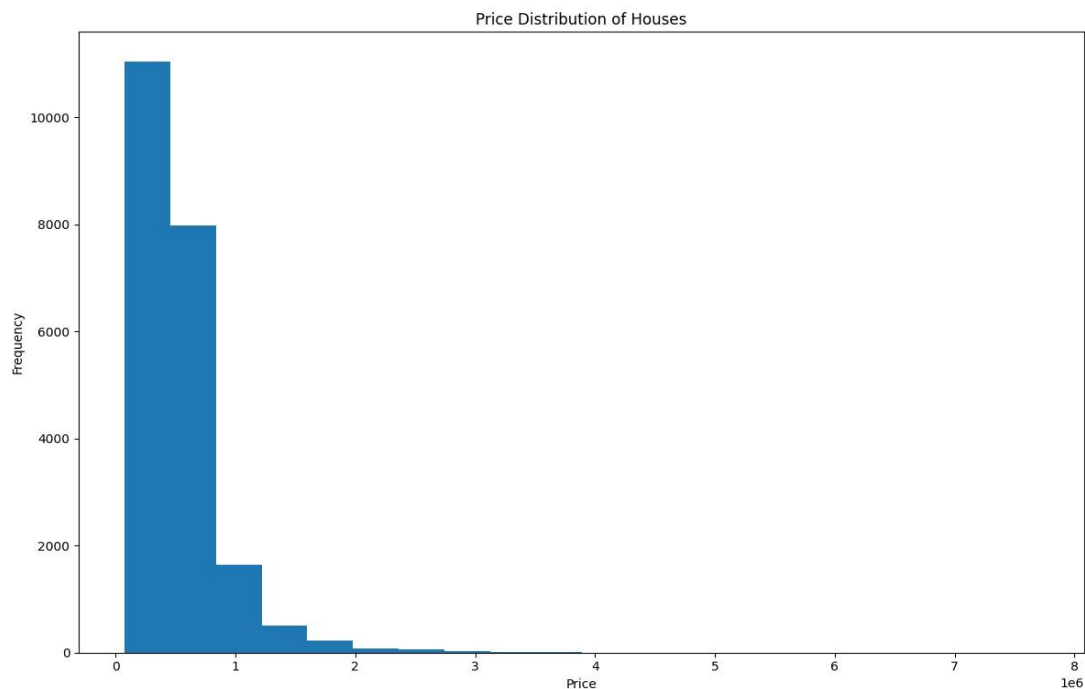


Fig 5.3: Histogram For Price Distribution of Houses.

Conclusion:

This provides a comprehensive understanding of the house dataset through advanced analysis techniques. Creating the Price Category feature helps in simplifying and grouping house prices effectively. The use of NumPy enables efficient analysis of price trends and variations. Visualizations such as line graphs, stacked bar charts, and histograms clearly present patterns and relationships in the data. The analysis shows that most houses fall in the mid-range category and prices are right-skewed. Overall, it highlights how data analysis can support better insights into real estate trends.

Final Conclusion:

The project demonstrates a complete data analysis workflow using the House Sales dataset. It begins with data loading and cleaning to ensure accuracy and reliability. Various visualizations like line graphs, bar charts, and pie charts help in understanding trends, distributions, and comparisons effectively. Filtering and grouping techniques highlight important patterns such as expensive houses and bedroom distributions. Advanced analysis using feature creation and NumPy provides deeper insights into pricing behavior. The results show that most houses fall in the mid-range category and price distribution is right-skewed.

What you have learned

Through this project, I learned how to perform complete data analysis using Pandas, NumPy, and Matplotlib. I understood how to clean and preprocess data by handling missing values and converting data types. I gained experience in creating different visualizations like line graphs, bar charts, and pie charts to analyze patterns. I also learned how to generate insights from data using grouping, filtering, and feature creation. Overall, this project improved my ability to interpret data and present meaningful conclusions.